safer, cleaner, and composable model for asynchronous code.

DATE : □□ - □□ - □□□□

Q1. Why Swift Concurrency? (async/await, Tasks, actors, structured concurrency)

- It replaces deeply nested callbacks and fragile thread-handling with language constructs that :-

(i) make async code look synchronous,
(ii) provide structured lifetimes for concurrent work (so tasks are not left running forever),
(iii) enforce data-race safety with actor and Sendable.

Q2. async/await -

- async :- marks a function that can suspend.
- await :- suspends the current task until that async call returns.

Note :- await suspends the Task, not a thread - the runtime can run other work on that thread.

Q3. Tasks : unstructured vs structured concurrency

Task (unstructured)

- Task {...} creates an unstructured child task that starts immediately and is not automatically tied to a caller's lifetime. (unless created from another task and not detached)

- Task.detached creates a task without inheriting parent task context (e.g., no priority inheritance, no actor context).

P.T.O.

DATE : □□ - □□ - □□□□

return try await withThrowingTaskGroup(of: Data.self) {group in
for url in urls{group.addTask {try await fetch(url)}}}

var results =[Data]()
for try await data in group{results.append(data)}}

return results

}
}

Q.4. Cancellation - cooperative model

- Cancellation is cooperative: calling task.cancel sets a flag. your code must check Task.isCancelled (or try Task.checkCancellation()) and return/cleanup.

- If you use APIs that respond to cancellation (many standard library and foundation APIs do), 'await'ing them may throw cancellationError.

```
func longWork() async throws {
    for i in 0...<1000{
        try Task.checkCancellation() //throw if
                                     //cancelled.
    }
}
let task = Task {
    try await longWork()
}
task.cancel()
```

signals cancellation.

DATE : □□ - □□ - □□□□

```
Task {
    await doWork()
}
```

```
Task.detached(priority: .background) {
    // runs independently
    await doBackgroundWork()
}
```

* Structured concurrency: async let and withTaskGroup.

Structured concurrency ties lifetimes: child tasks are scoped and automatically canceled when the parent scope exits (or when one fails in many cases), which prevents runaway background work.

```
async let sample' —
    func fetchAll() async throws {
        async let a = fetchA()
        async let b = fetchB()
    let (r1, r2) = try await (a, b) // runs concurrently
}
```

TaskGroup (use withTaskGroup / withThrowingTaskGroup) for dynamic concurrency and collecting: -

```
func fetchMany(urls: [URL]) async throws -> [Data] {
```

P.T.O.

DATE : □□ - □□ - □□□□

* "defer"

defer in Swift is a guaranteed cleanup block - it always runs at the end of the current scope, no matter how that scope exits.

```
Task {
    isLoading = true
    defer { isLoading = false }
    await fetch()
}
```

** What exactly happens?

1. Task start executing
2. isLoading = true runs immediately
3. Swift register the defer block.
4. await fetch() is executed.
5. When the Task completes - no matter success, failure, early return, or error - the defer block executes.
6. That sets isLoading = false

** Why use defer here?

Because it ensures your loading state is cleaned up even if "fetch()"

***

Summary

Keyword	Purpose
defer	Guarantees cleanup code runs when leaving scope
Runs	last, even after errors/returns
Use case	Manging loading states, closing files, removing observers, releasing locks.

UIKit GCD vs SwiftUI concurrency

DATE : □□ - □□ - □□□□

Concept	UIKit (GCD)	SwiftUI (Modern Swift)
Background work	DispatchQueue.global().async	Task{}, Task.detached
Update UI	DispatchQueue.main.async	MainActor auto-runs UI updates
Cancel work	WorkItem cancel	Task.cancel()
Parallel tasks	Concurrent queue	async let, TaskGroup
Wait for multiple tasks	DispatchGroup	withTaskGroup
Protect shared data	Serial queue, semaphore	Actors
Delayed execution	asyncAfter	Task.sleep

DATE : □□ - □□ - □□□□

Practical patterns & best practices

(i) Prefer async/await and structured concurrency (async let, withTaskGroup) for concurrency.

(ii) Use actor to protect mutable shared state; prefer immutable data when crossing concurrency boundaries.

(iii) Check Task.isCancelled or try Task.checkCancellation() for long work.

(iv) Use withCheckedContinuation to wrap legacy callback APIs (it helps you avoid resume mistake)

(v) Avoid blocking the executor threads (no heavy CPU bound sync work on the default executor — offload to Task.detached or DispatchQueue.global() when appropriate).

(vi) Use @MainActor for UI updates.

(vii) Favor Task {} over background DispatchQueue when you want cancellation propagation and structured semantics.

P.T.O.

Swift UI

Learn declarative
→ SwiftUI declarative style.

DATE : □□ - □□ - □□□□

* SwiftUI uses a declarative syntax, so you can simply state what your user interface should do. (SwiftUI is a declarative framework).
* "SwiftData" framework makes it easy to persist data using declarative tools.

@AppStorage : is a specific SWIFT-UI property wrapper that will use the users default under the hood.

property name

```
@AppStorage("onboarding") var isOnboardingViewActive: Bool = true
```

@AppStorage → property wrapper
"onboarding" → UserDefault key
true → value set

command to fold a long code block
option + ⌘ + ←

* SwiftUI runs on
  iOS 13 or any future later version.
  macOS 10.15 " " " " "
  tvOS 13 " " " " "
  watchOS 6 " " " " "
  visionOS 1 " " " " "

Async Image

DATE : □□ - □□ - □□□□

* In SwiftUI List, row height is calculated before AsyncImage finishes loading.

If the image uses scaledToFill or has unconstrained height, it expands vertically and overlaps adjacent rows.

Using resizable(), scaledToFit(), and a fixed frame prevents layout invalidation.

```
AsyncImage(url: URL(string: "abc.com")) { image in
        image
            .resizable()
            .scaledToFit()
            .frame(width: 80.0, height: 80.0)
            .clipped()
} placeholder: {
        ProgressView()
            .frame(width: 80.0, height: 80.0)
}
```

Note:- Having a little bit more padding to the horizontal direction than to the vertical direction is a well known practice amongst the UI designers.

DATE : □□ - □□ - □□□□

Button

```
Button(action: {
        print("Button tapped!")
})
{
    Text("Tap Me")
}
```

DATE : □□ - □□ - □□□□

Quick cheat-sheet

(i) async/await - suspend / resume
(ii) Task{} - unstructured child task
(iii) Task.detached - unscoped task.
(iv) async let - structured concurrency (simple)
(v) withTaskGroup / withThrowingTaskGroup - dynamic structured concurrency.
(vi) actor - serializes mutable state
(vii) Sendable - types safe to pass across concurrency boundaries.
(viii) @MainActor - UI/main thread isolation.
(ix) withCheckedContinuation - Wrap Callback APIs.
(x) Task.checkCancellation() - cooperative cancellation.

→ Combine is Apple's reactive programming framework introduced in iOS 13. It's designed to handle asynchronous events and data streams - like network responses, user inputs, or notifications - in a declarative way.

DATE : □□ - □□ - □□□□

Q. What is Combine and how does it relate to SwiftUI?

→ Combine is Apple's reactive framework that manages asynchronous data streams.

SwiftUI is build on top of Combine - when you mark a property as @Published in an ObservableObject, it becomes a Combine publisher.

SwiftUI subscribes to these publishers using @ObservedObject, @StateObject, or @EnvironmentObject, and automatically updates the view when values change.

Q. Does SwiftUI hold Combine subscriptions?

Yes, SwiftUI manages Combine subscriptions internally for property wrappers like @StateObject and @ObservedObject.
It automatically subscribes to published values and cancels the subscriptions when the view is destroyed.
However, if you manually create a publisher (like a Timer), you need to use onReceive or store the Cancellable yourself.

Q. What does "operator" mean in combine?

Think of an operator as:

"A tool that changes how values flow through a publisher.

$searchText
- debounce(...)
- removeDuplicates()
- sink { value in
    print(value)
  }

there!- $searchText → publisher
- debounce() → Combine operator
- sink() → subscriber

* debounce (used for "Typing/Search")

Wait until things stop changing, then act.

example!-
```
$searchText
  .debounce(for: .milliseconds(300), scheduler: RunLoop.main)
  .sink { text in
      print("Search for:", text)
  }
```

* throttle (used for Buttons/Scroll)

Allow only one value in a fixed time window.

Ex!- User taps button rapidly.
```
$buttonTap
  .throttle(for: .seconds(1), scheduler: RunLoop.main,
            latest: true)
  .sink { print("Button action triggered") }
}
// only one tap allowed per second.
```

Set :- a set is an "unordered" collection of unique element.

DATE : □□ - □□ - □□□□

Hashable

Hashable is a protocol. Any type that conforms to Hashable can be represent itself with a unique value called a hash.

Note :-
Hashable use Equatable as its base protocol. This means any type that attempts to conform to Hashable must also conform to Equatable.

Q. What is hashable?
→ A type that conforms to Hashable can be uniquely identified with a hash value.
→ Enable use in :
  * Set
  * Dictionary "key"
  * SwiftUI's forEach
→ Requires conformance to Equatable

*** Why use Hashable?

Feature	Purpose
Set	fast lookup, ensures uniqueness
Dictionary	Enables key-based storage and access
SwiftUI ForEach	Identifies views uniquely.
Performance	Optimized for hash table-based data structures.

DATE : □□ - □□ - □□□□

* How to Provide an id
  Two main approaches!
1. Use a property as id
   Example: forEach(names, id: \.self)
   Works if the element type conforms to Hashable.

2. Make the type conform to Identifiable
   struct User: Identifiable {
   var id = UUID()
   var name: String
   }

   forEach(users) { user in
   Text(user.name)
   }

*** Key Rule!-
→ For primitive types (Int, String, Double, etc.), you can just use \.self as id.
→ For custom types, prefer making them Identifiable.

Identifiable	Hashable
• Gives every item a unique ID	Gives every item a unique fingerprint.
• Used by SwiftUI Lists/forEach	Used for comparing, sets, dictionary keys.
• Needed for showing UI lists	Needed when using id:\.self

#Hashable is used for comparing objects.

↳ It creates a hash value (a no. that represents the object).
like creating a digital fingerprint.

Q. Why do we need hashable?
Because SWIFT uses hashing for :-

(i) Set
(ii) Dictionary Keys
(iii) fast comparisons
(iv) forEach(id:\.self)

DATE : □□ - □□ - □□□□

* identity vs identifiable

Feature	Identifiable	.id(_:)/View identity
What it is	A protocol to uniquely identify data models	A way to control view identity in SwiftUI.
Where used	Model layer (struct, class)	View layer (View modifiers)
Purpose	Helps SwiftUI know which data items are the same	Forces SwiftUI to treat a view as new or same.
Key Benefit	Efficient UI updates in lists/ForEach	Reset view state when needed

1. Identifiable Protocol :- it helps uniquely identify a value (struct, class, enum).
When a type conforms to it, it must have an id property that uniquely identifies each instance.

```
protocol Identifiable {
    associatedtype ID: Hashable
    var id : ID {get}
}
```

DATE : □□ - □□ - □□□□

* Example : Use in dictionary

```
let book1 = Book(title: "Swift", isbn: "123")
let library = [book1: "Shelf A"]
print( library[book1]!) // O/p : Shelf A
```

*** Best Practice :-

(i) Avoid mutable properties that affect hashing.
(ii) Ensure all stored properties are hashable.
(iii) For classes, remember NSObject conforms by default using reference identity.
(iv) Use id with Identifiable in SwiftUI alongside Hashable.

DATE : □□ - □□ - □□□□

* The actual type is known to the compiler, but hidden from the calling code.

```
func makeShape() → some Shape {
        return Circle()
}
```

- Returns a circle, but all the caller knows is that it's some Shape.
- You can use the returned value as a Shape, but you can't access Circle-specific API.
- It helps to keep things flexible and abstract.

Absolute and Relative position :-

DATE : □□ - □□ - □□□□

```
struct UserView: View {

@State var names = ["Paul", "Tom", "Laura"]

var body: some View {
        VStack {
            ForEach(names, id: \.self) { name in
                Text(name)
```

→ Here, SwiftUI iterates through the array and creates a Text for each name.
→ SwiftUI needs a stable identity (id) to tie each Text view to its corresponding name.
→ This ensures that when items are added/removed, SwiftUI knows which views to keep, update, or remove.

Identifiable + Hashable together

Why both :-

- Identifiable → for SwiftUI List.
- Hashable → for comparing, removing duplicates, storing in Set.

```
struct SignUpView: View {
@StateObject private var vm = SignUpViewModel()

    var body: some View {
        VStack {
            TextField("Email", text: $vm.email)
            SecureField("Password", text: $vm.password)
            Button("Sign Up", action: { /* call vm */ })
                .disabled(!vm.isValid)
        }
    }
}
```

```
import Combine

class SignUpViewModel : ObservableObject {
 @Published var email = ""
@Published var password = ""
@Published var isValid = false

private var cancellables = Set <AnyCancellable>()
// keep all combine subscriptions alive as long as this viewmodel
// exists.

 init() {
  Publishers.CombineLatest ($email, $password)
            .map { email, password in

        return email.contains ("@") && password.count >= 6
    }
    .assign (to: \.isValid, on: self)
    .store (in: &cancellables)
 }
}
```

---

* This ViewModel uses Combine's CombineLatest to observe email and password changes, validates them using a map operator, and assigns the result to an @Published property. SwiftUI observes that property and automatically enables or disables the button based on the validation state.

P.T.O.

* ObservableObject: This class has data that the UI should watch.
* @Published: Whenever this value changes → notify anyone who is listening.
* private var cancellables = Set<AnyCancellables>()
  Why do we need this?
  → Combine pipelines must be kept alive
  → If not stored → Combine stops listening.
* CombineLatest: Whenever email or password changes, give me both latest values.
* .map { email, password in
  return email.contains("@") && password.count >= 6
  } //returns true/false.
  checks if email is valid (contains "@") & password is long enough.
* .assign(to: \.isValid, on: self)
  → whatever result comes from above, put it into isValid.
  (This happens automatically, every time email or password changes).
* .store(in: &cancellables)
  Q. Why is this required?
  This says:- "Don't destroy this combine pipeline". Keep it alive.
  Without this line:- Validation would stop working.

*** Important property wrappers (SwiftUI + Combine) (ii)

(i) @Published var x - creates a publisher you can subscribe to (use in ViewModel).

(ii) @State - small local view state (owned by view)

(iii) @StateObject - use when the view creates/owns the ObservableObject.

(iv) @ObservedObject - use when the ObservableObject is passed in (not created by this view).

(v) @EnvironmentObject - global-ish dependency injected into environment.

*** Very common operators (what they do - one line)

(i) .map{} - transform values

(ii) .filter{} - pass only some values.

(iii) .removeDuplicates() - drop repeated values.

(iv) .debounce(for:, scheduler:) - wait for a pause (great for search)

(v) .throttle(for:, scheduler:, latest:) - limit frequency

(vi) .combineLatest(...)/Publishers.CombineLatest - combine two publishers' latest values.

(vii) .flatMap{} - switch into a new publisher (e.g. start an API call when input changes)

(viii) .sink(receiveCompletion: receiveValue:) - terminal subscriber to receive values/errors (returns AnyCancellable).

(ix) .assign(to: on:) or .assign(to:) - push publisher output directly to a property (returns AnyCancellable).

Combine (Beginner mental model)

"Think Combine as a set of channels that publish values over time. SwiftUI subscribes to those channels and redraws the UI when values change."

You rarely subscribe by hand for UI state - you mark data as @Published in a view model and bind that view model to your view; SwiftUI listens and updates the UI.

*** Cheat-Sheet

- Core pieces (super-simple)

(i) Publisher - produces values (e.g., $username, Just, URLSession.DataTaskPublisher)

(ii) Subscriber - receives values (SwiftUI does this for you when you bind view to a view model)

(iii) Cancellable/AnyCancellable - keeps a subscription alive and allows cancellation. Store cancellable(s) in a Set<AnyCancellable>().

*** Combine provides three big categories of things :-

1. Publishers → who sends values
2. Operators → how values are transformed/controlled.
3. Subscribers → who receives values.

* Combine in one sentence

- Combine lets your app react automatically when data changes.

[OR]

Combine connects your data to your UI automatically.

If data changes → UI updates

What Combine fixed? (Why it was required)

Before Combine	With Combine
1. Delegate	Publisher
2. Closure	Sink
3. Notification	Subject
4. Timer	TimerPublisher
5. KVO	@Published
6. GCD hacks	debounce/throttle

** Debounce and throttle

- debounce and throttle are part of the Combine framework. They are Combine operators, not separate concepts outside Combine.

P.T.O.

(c) Combining (very common) ***

• CombineLatest - latest values from all publishers

• zip - pair values in order

• merge - merge same-type streams

• switchToLatest - use newest publisher

(d) Timing

* • debounce - wait for pause (search typing)

* • throttle - limit frequency (button taps)

• delay

• timeout

(e) Threading

• receive(on:) - UI thread

• subscribe(on:) - background work.

(iii) Subscribers - Who receives data

• sink - handle values & completion

• assign - assign output to property

• SwiftUI - auto-subscribes to @Published.

* CombineLatest (Key operator)

combineLatest observes multiple publishers and emits their latest values whenever any changes.

Used for :-

✓ form validation

✓ multiple field dependencies

✓ state derivation

#Memory management

• Combine pipelines must be stored

• Use Set<AnyCancellable>

• .store(in: &cancellables)

• When ViewModel deallocates → subscriptions cancel automatically.

DATE : □□ - □□ - □□□□

* Quick x Mnemonics

@State → I own
@Binding → I borrow
@ObservedObject → I observe (but don't own)
@StateObject → I create + observe
@EnvironmentObject → I share globally.

@State vs @StateObject.

@State
(i) Used for simple value types (String, Int, Bool, struct, etc)
(ii) The view owns and manages the data directly.
(iii) SwiftUI stores it outside the view struct so it survives re-renders.
Use Case :— local UI state.

@StateObject
(i) Used for reference types (classes conforming to ObservableObj.)
(ii) The view owns the object and creates it only once for its lifetime.
(iii) Prevents the object from being recreated when the view re-renders.

.firstIndex(of:)

DATE : □□ - □□ - □□□□

```
var gussedLetters = [ Letter(id: 1, text: "A"),
                      Letter(id: 2, text: "B")
                    ]
let gussedLetter = Letter(id: 2, text: "B")
let index = gussedLetters.firstIndex(of: gussedLetter)
```

O/P : index = 1.

* .firstIndex(of:) returns the position (Int) of the first matching element in the array.

Q. What is Identity in SwiftUI?
** → Identity is essential in SwiftUI because many views (e.g., forEach, List) require an id to track and manage elements.
→ SwiftUI views are structs, and the entire value of a view is re-evaluated whenever state changes.
→ Identity allows SwiftUI to know which view corresponds to which data element when re-rendering.

Q. Why is Identity Needed?
→ Without identity, SwiftUI wouldn't know if a Text("Paul") is the same as before or a new element.
→ Stable ids prevent SwiftUI from unnecessarily destroying and recreating views.

DATE : □□ - □□ - □□□□

* Compiler Synthesis (Swift 4.1)

You get == and hash(into:) for free if:

→ All stored properties are themselves Hashable.

```
struct Person: Hashable {
    let name: String
    let age: Int
}
```

* Custom Implementation

If needed, you can override:

```
struct Book: Hashable {
    let title: String
    let isbn: String

    func hash(into hasher: inout Hasher){
        hasher.combine(isbn)
    }

    static func ==(lhs: Book, rhs: Book) -> Bool {
        return lhs.isbn == rhs.isbn
    }
}
```

Example in Set :-

```
let a = Person(name: "Aseem", age: 28)
let b = Person(name: "Geetika", age: 25)
```

```
let x: Set = [a, b]
```

P.T.O.

DATE : □□ - □□ - □□□□

* Cancellation propagation: structured child tasks inherit cancellation from parent; with Task.detached they don't.

5. @MainActor, Task priorities, and executors

@MainActor
- Annote functions or types to ensure they run on the main actor (main thread UI work).

```
@MainActor func updateUI() {...}
```

Calling such methods from background tasks requires await (if crossing actor boundaries).

* Opaque return types hides the concrete type of whatever you return.
It can be a protocol, class, struct or enum.
But the return must always be one specific concrete type.

DATE: □□-□□-□□□□

Q. What is an opaque return type in Swift?
A return type "concrete type" where the function hides its conformance using "some"

Q. How is some different from any?
- some → some concrete type, compile-time known
- any → any type conforming to protocol, runtime known.

Q. Why is "some View" required in SwiftUI?
Because View is a protocol with associated types, cannot be used as a return type directly.

Q. Can a function returning some View return different view types?
No. All branches must return the same concrete type.

Q. Do opaque types preserve type information?
Yes, compiler knows the exact type; callers do not.

Q. Do Can Opaque types appear in stored properties?
Only in Swift 5.7+
Must be initialized immediately.

Ex1-

```
let shape: some Shape = Circle()
```

Combine - Quick Notes (SwiftUI)

* Combine is apple's reactive framework that lets you publish values over time and react to them automatically.
(SwiftUI uses combine internally to update UI when data changes)

** Core Building Blocks

(i) Publishers (ii) Operators (iii) Subscribers

(i) Publishers - Who sends data
(*) @Published - publishes when a property changes
(*) Just - emits one value
(*) PassthroughSubject - manually send values
(*) CurrentValueSubject - stores + emits latest value
(*) URLSession.DataTaskPublisher - API calls
(*) Timer.TimerPublisher - time-based events.

(ii) Operators - What happens to data
(Operators transform, filter, combine, or control timing)

(a) Transform
- map
- flatMap
- scan

(b) Filtering
- filter

DATE : □□ - □□ - □□□□

// Simple example of @propertyWrapper without initial value (init())

```
@propertyWrapper
struct NonNegative {
    private var value : Int
    var wrappedValue : Int {
        set { value = max(0, newValue) }
        get { value }
    }
}
```

ex/

```
struct User {
    @NonNegative let age : Int
    init (age : Int) {
        self.age = age
    }
}

let u1 = User (age : -10)
let u2 = User (age : 2)
print (u1.age)  // 0
print (u2.age)  // 2
```

DATE : □□ - □□ - □□□□

* Property Wrapper ***

A property wrapper is a way to add extra behaviour to a property without repeating code.

Note :-
Extensions "adds reusable behavior to a type, while property wrappers add reusable behaviour to individual properties. They solve different problems and are complementary, not interchangeable."

*** "Property wrappers allow us to encapsulate reusable property logic like validation, transformation, or state handling, and apply it declaratively using the @ syntax, improving code reuse and readability."

```
@propertyWrapper
struct NonNegative {
    private var value: Int

    init(wrappedValue: Int) {
        self.value = max(0, wrappedValue)
    }

    var wrappedValue: Int {
        get { value }
        set { value = max(0, newValue) }
    }
}
```

P.T.O.

DATE : □□ - □□ - □□□□

Q. What is a property wrapper?

A property wrapper is a swift feature that lets you add common behaviour (logic) to properties without repeating code.

"A reusable box that adds extra behaviour to any property"

Simple example :-

```
@propertyWrapper
struct Uppercased {
    private var value: String = ""

    var wrappedValue: String {
        get { value }
        set { value = newValue.uppercased() }
    }
}

struct User {
    @Uppercased var name: String
}

var u = User()
u.name = "aseem"
print(u.name)   // "ASEEM"
```

* A property wrapper is created using @propertyWrapper, and it must expose a wrappedValue property that defines how the wrapped property is stored and accessed.

DATE : □□ - □□ - □□□□

* @StateObject , @ObservedObject , @EnvironmentObject.

@StateObject
→ Used when a view creates & owns an ObservableObject.
→ Initialized once per view lifecycle (survives redraws)
→ Acts as the single "source of truth."

Ex!-

```
@StateObject var viewModel = MyViewModel()
```

@ObservedObject
→ Used when a view receives an existing ObservableObject.
→ The view only observes changes; does not own lifecycle.
→ Typically used in child view.

Ex!-

```
@ObservedObject var viewModel: MyViewModel
```

@EnvironmentObject
→ Used to inject and access an ObservableObject through the SwiftUI environment.
→ Allows sharing data across many views without passing explicitly.
→ Must be set with .environmentObject(_:) at a higher level.

Ex!- @EnvironmentObject var session: UserSession.

Rule of thumb
→ @StateObject → Create & own
→ @ObservedObject → Pass down & observe
→ @EnvironmentObject → Share globally.

#Blackbox Programming.

Page TabView
@State
@AppStorage

DATE : 12 - 07 - 2025

* @main :- designates the entry point of the application. (SwiftUI, uses this instead of the traditional "main.swift" file found in UI-Kit based app).

* var body: some Scene {
↳ every SwiftUI app must define a body that returns one or more Scenes.
↳ Think of a Scene as a container for parts of your UI (like windows or views).
↳ Its hierarchy of views.

* WindowGroup {
↳ WindowGroup is a type of Scene that represents a group of app windows.
↳ On iOS, it usually means a single app window; on macOS/iPadOS, it can mean multiple windows.
↳ It creates the main window of the app.

Q What is an Opaque Return Type?
→ An opaque return type in Swift is declared using the "some" keyword in a function's return type:-

func makeSomething() → some Protocol.

↳ This function returns a specific, concrete type that conforms to the given protocol, but I'm not exposing the exact type to the caller.

P.T.O.

In 2-way binding, we use updateUIView()

DATE : □□ - □□ - □□□□

UIViewRepresentable

UIViewRepresentable is a SwiftUI protocol that allows you to integrate and display UIKit views inside SwiftUI.

Purpose :- (When used)
→ A SwiftUI equivalent doesn't exist.
→ You want to reuse existing UIKit components.
→ You need to access UIKit-specific features or delegates.

Required methods :-
1. makeUIView(context:) → Creates & configures the UIKit view. (called once)
2. updateUIView(_:context:) → Updates the view when SwiftUI's state changes.

Optional methods :-
1. makeCoordinator() → Creates a bridge (coordinator) for delegate & data communication b/w UIKit & SwiftUI

Life Cycle :-
- SwiftUI → creates the view via makeUIView
- SwiftUI → updates view whenever state changes via updateUIView

When to use Coordinator :-
- When the UIKit component uses delegate methods (e.g. - UITextfieldDelegate, UIScrollViewDelegate).

DATE : □□ - □□ - □□□□

ViewBuilder

- It's a result builder that combines multiple child views in a closure into a single view hierarchy.
- It enables SwiftUI's declarative syntax.

OR

ViewBuilder is a function builder (now called result-builder) used by SwiftUI to let you build multiple view declaratively inside closure, like in VStack, HStack or body.

source-code

```
struct MyContainer<Content: View>: View {
    let content: Content
    init(@ViewBuilder content: () -> Content){
        self.content = content()
    }

    var body: some View {
        VStack(spacing: 10){
            content
        }
    }
}
```

"@ViewBuilder" is a special attribute that tells SwiftUI how to combine multiple views into a single Tuple View.